प्रमाण

PRAMAAN *evidence*

Getting better and worse at the same time

A document-grounded claim checker for Nepali equities — and four measured iterations that never improved the score I was watching.

Dipendra Limbu · micro1 Frontier Engineering Challenge 2026
github.com/DKL59/Micro1_Challenge

Problem Baseline Method One execution Comparison What mattered Hot take

1 The problem

Synthetic — written as a test input, modelled on what circulates publicly. Every number in it is real.

> *"Nabil declared 30% dividend this year. Bank FD is giving only 4%. Why keep money in fixed deposit when a blue chip bank is paying 30%? Long term hold, guaranteed income."*

WHAT THE SOURCES ACTUALLY SAY

Nabil did declare 30.00% — for fiscal year 2078/79, three years earlier (18.50% bonus + 11.50% cash).

For the year the claim is about, 2081/82, Nabil declared 12.50% cash and no bonus.

A dividend percentage is calculated on the Rs 100 par value — not on the Rs 540.20 the share actually costs.

THE ARITHMETIC THAT DECIDES IT

`Rs 12.50  ÷  Rs 540.20`

2.31%

dividend yield on money actually invested

2.75% – 4.55%

individual fixed deposit rates across the three banks in the sources

Both sides are what money invested today earns: the latest declared dividend against the rates on offer now. A past deposit rate cannot be bought — and unlike the dividend, the deposit rate is contractual.

Every number is true. They support the opposite conclusion.

SEBON named unauthorised investment advice a live harm in July 2026 — which is also why these cases are synthetic. A real post would identify someone now exposed to enforcement.

2 Who this is for

The intended user, the bottleneck they hit, and why closing it is worth doing

THE USER

Someone deciding whether to act on an investment claim about a listed Nepali company.

The figures in the claim may well be real. What is missing is a fast way to see where each one comes from, and whether it supports the conclusion drawn from it.

THE BOTTLENECK

Checking that single claim by hand means:

1. find Nabil's dividend history on ShareSansar
2. find the current market price
3. pull fixed deposit rate cards from three banks
4. know that a declared dividend is a percentage of the Rs 100 par value, not of what the share costs

Step 4 is the one that decides it. Miss it and you reach a confident, wrong answer without ever noticing.

WHY IT IS WORTH CLOSING

The failure here is not ignorance. It is confident arithmetic on the wrong denominator — which survives any check that only asks whether the numbers are real.

SEBON named unauthorised investment advice a live harm in July 2026, with licensing requirements and legal action planned — so the value of the problem is evidenced rather than asserted.

WHAT A USER WOULD GET

An answer where every figure arrives with the sentence it came from, so it can be checked rather than trusted — and an explicit “not covered by these sources” when the documents do not settle it, instead of a guess. It never tells anyone to buy or sell.

3 The simple baseline

baseline.py — one API call, one line of instruction, no documents at all

WHAT IT GOT RIGHT

- Reached the correct verdict on all three cases
- Knew a dividend is quoted on par value, unprompted
- Rejected “guaranteed income” on its own
- Explained why bonus shares are value-neutral

WHAT IT INVENTED

```
“you might have to pay a Market Price
 of NPR 600 or more”

“your actual return on investment (yield)
 might only be 3% to 5%”
```

Both fabricated. It concluded the yield was “very close to the 4% FD rate”. The real figure, 2.31%, sits below the floor of the range.

3 / 3

verdict agreement — with no documents

0 / 3

figures traceable to a source

1,832

mean tokens per case · 15.8s

4 Document-grounded claim verification

It reads only the documents it is handed. Nothing is retrieved from the web — a real limit, and the reason every run reproduces exactly.

1 · INPUT

The claim, plus a fixed corpus of published filings — each with its URL and a line stating what that file does and does not cover.

2 · CLAIM DECOMPOSITION

The claim is broken into its separate assertions. A claim can be partly true, and this is how that becomes visible.

3 · VERDICT ASSIGNMENT

Every assertion is judged against the corpus alone. General knowledge about these companies is forbidden.

4 · CITATION ENFORCEMENT

No figure may be reported without the exact sentence it was taken from, and the file that sentence lives in.

5 · SCHEMA VALIDATION + CORRECTION LOOP

Three mechanical rules check the response. A failure is returned with the violation named, up to three attempts.

THE VERDICT SET — THREE VALUES, NOT TWO

`verified`

The sources support it. The quote that settles it is attached.

`contradicted`

The sources say otherwise. The quote that contradicts it is attached.

`not_found`

The sources do not settle it. Not “false” — outside what these documents cover. Each file states its own coverage, so this is a claim about a known boundary rather than a shrug.

A model forced to answer yes-or-no guesses when it does not know — which is exactly how the baseline invented a share price. not_found is the escape hatch that stops it bluffing.

It never advises buying or selling.

HOW AGENTIC IS THIS, HONESTLY? Stage 5 is the only part that acts, checks its own output and acts again — the minimum that qualifies. No planning, no tool use, no memory. And the control run shows that loop bought nothing measurable.

5 One execution, start to finish

Case 3 · EBL undervaluation · results/agent_v4.json — the challenging case

"EBL net profit up 32% this quarter, best in the sector. Still trading below book value, market has not priced it in yet. Undervalued blue chip, accumulate now."

CASE 1 · NABIL

1 attempt · clean first time

CASE 2 · NICA

2 attempts · 3 citations repaired

CASE 3 · EBL

2 attempts · 1 repaired · shown below

THE RUN

5 API calls · 3 cases · 0 violations

ATTEMPT 1

The agent finds the 32% — and finds it belongs to the whole commercial banking sector, not to EBL. Marks the assertion contradicted and quotes the line.

`6,417 tokens`

→

VALIDATOR

Rejects the response. Not the verdict — the citation.

assertion 1: quote does not appear in sources/ebl.md

`1 violation`

→

ATTEMPT 2

The same response is returned with the violation named and an instruction to fix that and change nothing else. Comes back clean.

`5,758 tokens`

Every attempt is stored in the result file with its own token count. The correction is in the evidence rather than hidden behind a tidy final answer.

6 What actually changed between the two attempts

Found on the last day, while writing the video script — and documented rather than fixed

ATTEMPT 1 — REJECTED

```
"Commercial banks' combined net profit rose 32.33% to NPR 69.78 arba in Q4 2082/83."
```

ATTEMPT 2 — ACCEPTED

```
"Commercial banks' combined net profit rose 32.33% to NPR 69.78 arba in"
```

sources/ebl.md — LINES 80 AND 81

```
Commercial banks' combined net profit rose 32.33% to NPR 69.78 arba in ⏎
Q4 2082/83. Nabil Bank is reported as leading the sector.
```

Same verdict. Same figures. Same summary. The only difference is that the quote now stops at the line break.

agent.py hands the validator the file as written. validate.py hands it the same file with whitespace collapsed.

The model was not learning to cite better. It was learning where the file wraps.

7 The final comparison

Five runs plus a control — one variable changed per iteration

Run	What changed	Verdicts	Schema violations	Mean time	Mean tokens
Baseline	No documents at all	3 / 3	—	15.8s	1,832
Iteration 1	Grounding + structured output	3 / 3	5	16.4s	5,338
Iteration 2	My conclusions stripped from sources	2 / 3	2	15.3s	4,805
Iteration 3	Range and ratio rules in the instruction	3 / 3	4	20.3s	5,620
Iteration 4	**Validation loop — shipped**	**3 / 3**	**0**	**70.1s**	**10,741**
Control (v5)	Same instruction, loop disabled	3 / 3	0	20.7s	6,373

A schema violation is a response that breaks one of the validator's three rules: a status outside the three permitted values, a quote that does not exist in the file it names, or a figure whose numbers are absent from its own quote. The verdict can still be right — it just cannot be checked.

WHAT NEVER MOVED

Verdict agreement: 3, 3, 2, 3, 3. The baseline — with no documents at all — scores the same as the shipped agent.

BIGGEST CHANGE FROM THE BASELINE

Figures traceable to a source: 0/3 → 3/3, from Iteration 1. Attribution error: 0/1 → 1/1 — but unaided only from Iteration 2, once I stopped planting the answer in the corpus.

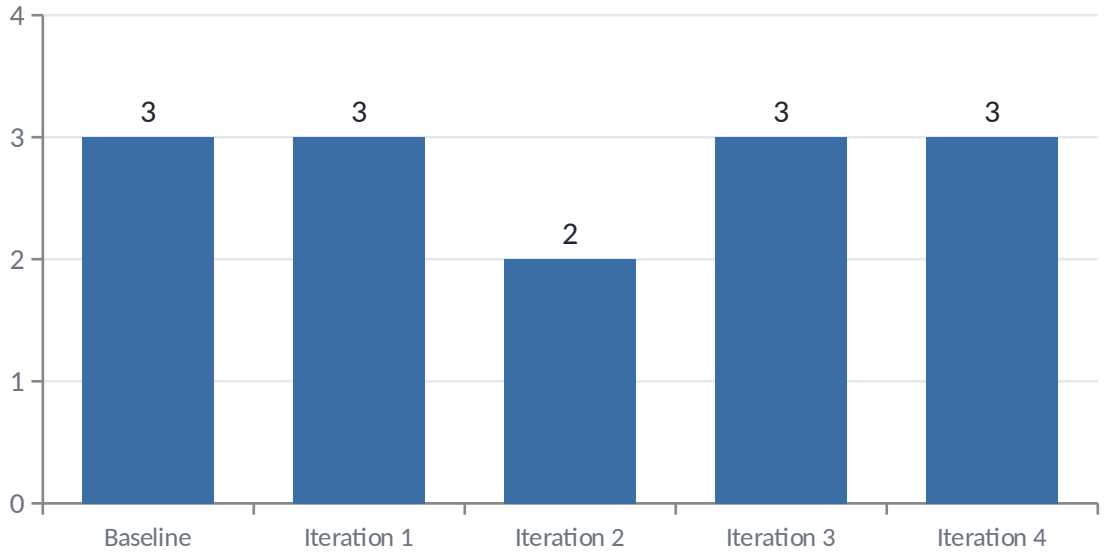
BIGGEST CHANGE ACROSS THE ITERATIONS

Schema violations: 5 → 0. Not a baseline comparison — free text has no schema to break. And the control scores 0 with the loop off, so the instruction rules did this, not the loop.

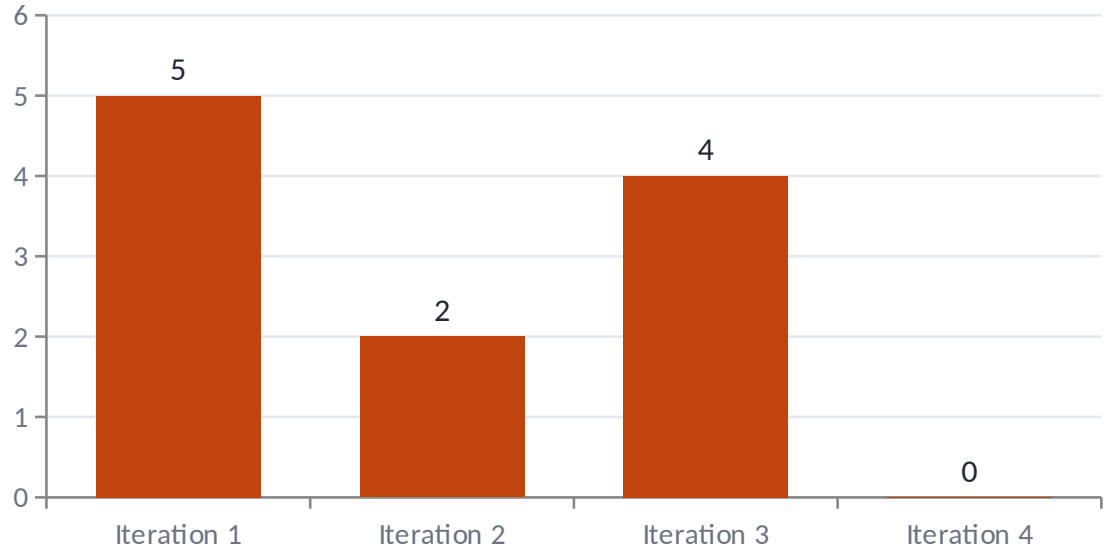
8 The axis I was not watching

Two measures, shown separately because they are different scales — and the baseline appears on only one of them

Verdict agreement (out of 3) — baseline included



Schema violations (lower is better) — no baseline bar



Left: the baseline, with no documents at all, matches the shipped agent. That is the ablation. Right: it has no bar, because free text has no schema to break — which is why 5 → 0 is not a comparison against the baseline. Iteration 2 is my worst run on verdicts and my best on citations; Iteration 3 fixed a real reasoning defect and doubled the violations. These four runs produced eleven violations in total, and ten of them are citation failures.

9 The changelog

One variable per iteration, each with a prediction written before the run

BASELINE	What a user gets today: paste the claim into a chat window.	*Reasons well. Verifies nothing.*
ITERATION 1	Gave it the published figures; required a quote and file per figure.	*Traceability 0/3 → 3/3. But I had written my own conclusions into the sources — so the attribution catch measured retrieval, not detection.*
ITERATION 2	Stripped my analysis out of the sources into notes/. Only published figures remain.	*The finding survived unaided. Cost one verdict — and that is the run that made every later number mean something.*
ITERATION 3	Two rules added: state the full range when comparing; compute any ratio the claim turns on.	*3/3 restored and earned. Citations got twice as bad.*
ITERATION 4	Built validate.py and wired it into the agent: fail → return the violations → retry.	*Schema violations to zero. 3.5× the time and 1.9× the tokens of Iteration 3.*

10 What contributed most — and one experiment I removed

NOT THE VALIDATION LOOP

A control run with the loop disabled scores identically on both machine-checked metrics.

	Loop on	Loop off
Verdicts	3 / 3	3 / 3
Violations	0	0
Mean time	70.1s	**20.7s**
Mean tokens	10,741	**6,373**

IT WAS ITERATION 2

I had written my own conclusions into the source files the agent reads — so it was scoring well by retrieving answers I had planted. I stripped them into a folder it never sees and ran it again. It caught the attribution error anyway.

That run made every later number mean something.

THE EXPERIMENT I REMOVED

Discrimination

I had recorded a third weakness in the baseline: that it condemns a whole claim rather than separating the accurate part from the misleading part.

It rested on an assumption — that Nabil's 30% was real for the current year. During a file audit I checked the source. It was not: Nabil declared 12.50% for 2081/82.

The gap dissolved. I removed it rather than leave a conclusion standing on a premise I never verified.

A figure taken on trust and never traced to its source — the exact failure this project was built to catch.

11 What this does not establish

Stated here because the project's entire argument is that unchecked claims are the problem

THREE EVALUATION CASES · ONE ATTRIBUTION CASE

A case study, not statistical power. The free tier allows twenty requests a day, and six runs of three cases is eighteen. The attribution result rests on a single case — the weakest evidence here.

NO USER HAS USED IT

It ships as an evaluation harness, not a product. The README says so in its first paragraph. Nothing here shows anyone would find it useful in the wild.

THE SCORED METRIC NEVER SEPARATED THE SYSTEMS

Verdict agreement was the wrong thing to optimise. I only know that because I measured it and published the result.

WHAT YOU CAN CHECK YOURSELF

Five defects were found in this system, documented, and shipped unfixed — because tightening a rule on the last day would re-score five completed runs against a contract the agent was never given. The reasoning for each is dated in DECISIONS.md.

FOUR SCRIPTS · NO API KEY · NO NETWORK

```
check_results.py     timings, tokens, verdicts
validate.py          schema violations per run
test_validate.py     23 tests over the validator
check_divergence.py  the line-break finding
```

19 dated decision records · 7 result files, never hand-edited or overwritten · a failed run kept because a failed run is evidence too · full git history in the repository

HOT TAKE

The axis you aren't measuring is the one your agent is moving on.

The fix is not “measure more things” — you cannot act on that. It is narrower, and it is uncomfortable:

Remove the part you're proudest of, and run it again.

Take your documents away and see whether your score survives. Mine didn't.

`github.com/DKL59/Micro1_Challenge  ·  19 recorded decisions  ·  23 tests, no API key`